

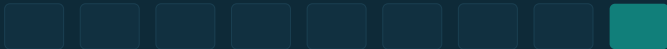


STREAM PROCESSING AVEC APACHE KAFKA

# Kafka Connect

Intégrer des sources et des sinks externes — sans écrire de code

ESGI · 5<sup>e</sup> année · Data Engineering · 2025–2026



# Objectifs

- **Comprendre quand utiliser Connect**

Déplacer des données entre Kafka et des systèmes externes, sans producteur/consommateur sur mesure.

- **Connaître l'architecture**

Workers, connecteurs, tasks, convertisseurs, transformations (SMT).

- **Piloter Connect par son API REST**

Déployer un connecteur = envoyer une configuration JSON, sans code.

- **Choisir le bon convertisseur**

String, JSON, ou Avro — ce dernier réutilise le Schema Registry de la séance 7.

## SÉANCE 8

# Au programme

01

### **Pourquoi & quand**

intégration sans code

02

### **Architecture**

workers, tasks, REST

03

### **Convertisseurs & transforma- tions**

String, JSON, Avro, SMT

04

### **Atelier**

source, sink, SMT

# 01

- PARTIE 1

## Pourquoi & quand

# Arrêter d'écrire des intégrations jetables

- Connecter Kafka à une base, un fichier, un service... revient souvent à réécrire le **même** code producteur/consommateur (lecture, sérialisation, reprise sur erreur, parallélisme).
- **Kafka Connect** industrialise ce travail : on *déclare* une intégration (en JSON) au lieu de la *coder*.

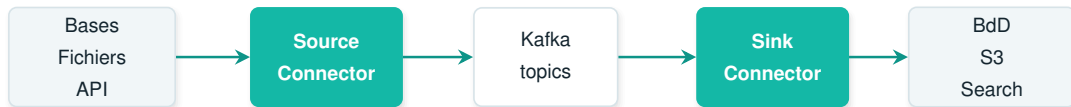
## ■ Sans Connect

Un projet à maintenir par source et par destination ; reprise et montée en charge à refaire.

## Avec Connect

Des connecteurs réutilisables,  
scalables et tolérants aux  
pannes, pilotés par configuration.

# Sources et sinks autour de Kafka



## ■ Source

Lit un système externe et **publie** dans des topics Kafka.

## ■ Sink

**Consomme** des topics et écrit vers un système externe.

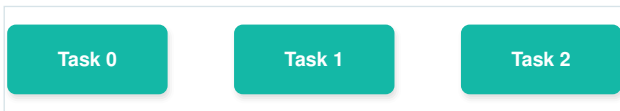
# 02

- PARTIE 2

## Architecture

# Workers, connecteurs, tasks

Connecteur (1 configuration)



tasks réparties sur les workers (parallélisme)

topics internes  
configs · offsets · status

- Un **worker** est un processus JVM ; en **mode distribué**, plusieurs workers forment un cluster qui se partage les tasks et rééquilibre en cas de panne.
- L'état (configurations, offsets, statuts) vit dans des **topics Kafka internes** : le cluster est sans état local.



# Un connecteur = une configuration JSON

Un nom, une classe de connecteur, et ses paramètres :

`file-source.json` - soumis à l'API REST

```
{
  "name": "file-source",
  "config": {
    "connector.class":
      "org.apache.kafka.connect.file.FileStreamSourceConnector",
    "tasks.max": "1",
    "file": "/data/orders.txt",
    "topic": "orders-connect",
    "value.converter":
      "org.apache.kafka.connect.storage.StringConverter"
  }
}
```

# Déployer une source via REST

Alimenter un fichier, deployer le connecteur, lire le topic

```
printf 'o-1;19.9\no-2;5.0\n' > data/orders.txt    # 1 ligne = 1 commande
curl -s -X POST -H "Content-Type: application/json" \
  --data @connectors/file-source.json localhost:8083/connectors
curl -s localhost:8083/connectors/file-source/status    # RUNNING ?
kafka-console-consumer.sh --bootstrap-server localhost:29092 \
  --topic orders-connect --from-beginning --max-messages 2
```

## ■ RUNNING

Connecteur et tasks à l'état RUNNING.

## ■ Sans code

Les lignes arrivent dans  
orders-connect.

# 03

- PARTIE 3

## Convertisseurs & transformations

# Du format interne aux octets Kafka

Le convertisseur (dé)séréalise entre le connecteur et Kafka :

## Convertisseurs cle / valeur (extrait de config)

```
"key.converter": "org.apache.kafka.connect.storage.StringConverter",  
"value.converter": "io.confluent.connect.avro.AvroConverter",  
"value.converter.schema.registry.url": "http://localhost:8081"
```

### ■ String / JSON

StringConverter, JsonConverter  
: simples, sans registre.

### Avro + registre

**AvroConverter** réutilise le  
Schema Registry de la séance 7.

# Single Message Transforms (SMT)

Modifier chaque message à la volée, toujours sans code :

## Renommer le topic cible avec RegexRouter

```
"transforms": "route",  
"transforms.route.type":  
  "org.apache.kafka.connect.transforms.RegexRouter",  
"transforms.route.regex": "(.*)",  
"transforms.route.replacement": "routed-$1"
```

Autres SMT courants : `InsertField`, `MaskField`, `ReplaceField`, `ValueToKey` (sur données structurées).

# Un sink qui réécrit vers un fichier

## Deployer le sink, observer le fichier de sortie

```
# Sink FileStream : le topic est reverse dans un fichier
curl -s -X POST -H "Content-Type: application/json" \
  --data @connectors/file-sink.json localhost:8083/connectors
# Le fichier de sortie reflète le contenu du topic
cat data/orders-out.txt
# Lister les connecteurs actifs
curl -s localhost:8083/connectors      # ["file-source", "file-sink"]
```

### ■ Aller-retour

Fichier → topic (source) → fichier (sink),  
sans une ligne de code.

### ■ Découplage

Le même topic pourrait alimenter plusieurs  
sinks en parallèle.

# 04

- PARTIE 4

## **Atelier — un pipeline Connect de bout en bout**

# Cluster + registre + worker Connect

## Demarrer la pile et verifier les plugins disponibles

```
docker compose up -d          # 3 brokers + schema-registry + connect

# Le worker Connect repond sur 8083
curl -s localhost:8083/
# Lister les plugins installes (connecteurs + transformations)
curl -s localhost:8083/connector-plugins
```

### ■ Worker

cp-kafka-connect branché sur les brokers, REST sur localhost:8083.

### ■ Plugins

FileStream, convertisseurs et SMT sont déjà présents (rien à installer).



# Source, sink, transformation

- 1 Déployer le connecteur **source** (fichier → `orders-connect`) et vérifier `/status`.
- 2 Consommer `orders-connect` pour confirmer l'arrivée des lignes.
- 3 Déployer le connecteur **sink** (`orders-connect` → fichier) et lire la sortie.
- 4 Ajouter une ligne au fichier source : elle traverse tout le pipeline.
- 5 Ajouter un **SMT** (`RegexRouter`) à la source et observer le nouveau topic.

# Piloter par l'API REST

## Suivre l'état et la configuration des connecteurs

```
curl -s localhost:8083/connectors # liste
curl -s localhost:8083/connectors/file-source/status # état + tasks
# Mettre à jour une config à chaud (idempotent) : PUT
curl -s -X PUT -H "Content-Type: application/json" \
  --data @connectors/file-source.json \
  localhost:8083/connectors/file-source/config
```

### ■ Tout par REST

Lister, inspecter, mettre à jour, supprimer — sans redémarrage.

### ■ Idempotent

PUT .../config crée ou met à jour le connecteur sans rien casser.

# Pour aller plus loin

**Objectif** — relier Connect au Schema Registry et durcir le pipeline.

## ■ 1. Avro

Configurer un sink en `AvroConverter` vers le registre de la séance 7.

## ■ 2. Router

Enchaîner deux SMT : `RegexRouter` puis un filtre, et observer.

## ■ 3. DLQ

Activer `errors.tolerance` + une dead letter queue sur le sink.

# L'écosystème Connect

- **Des centaines de connecteurs** prêts à l'emploi : JDBC, Debezium (capture de changements de bases), S3, Elasticsearch... (Confluent Hub).
- **MirrorMaker 2** est bâti sur Connect : réplication de topics entre clusters.
- **En production** : montée en charge par `tasks.max` et workers, dead letter queues, supervision via l'API REST et JMX.

## ■ Intégrer

Connect = la colle entre Kafka et le reste du système d'information.

## ■ Transformer

Pour la logique riche sur les flux : Kafka Streams (arc suivant).

## SÉANCE 8

# À retenir

- ✓ Connect déplace des données entre Kafka et l'extérieur, **par configuration** et non par code.
- ✓ Sources (vers Kafka) et sinks (depuis Kafka) ; un connecteur se découpe en **tasks**.
- ✓ Mode distribué : état dans des topics internes, piloté par l'**API REST** (port 8083).
- ✓ Le **convertisseur** choisit le format ; `AvroConverter` réutilise le registre de la séance 7.
- ✓ Les **SMT** transforment message par message ; la logique riche, c'est Kafka Streams.



LA SUITE — SÉANCE 9

# Kafka Streams

Traiter et transformer les flux dans l'application : KStream, KTable, et la topologie de traitement.